

1st Deliverable (Design a relational database)

Submission By: Shikha Raj(sraj11@hawk.illinoistech.edu)

Selected Application: Spotify (<https://open.spotify.com/>)

Report content:

1. Description of Spotify users and their needs
2. ERD of spotify database
3. Relational/Logical database Schema Representation of Spotify
4. Spotify Database Tables: Data Variables, Semantics, and Constraints
5. User Interaction Use Cases for the Spotify Database

Spotify Users and their needs

1. Listener: Person who uses Spotify to listen to music or podcasts. A listener can have different kinds of accounts (Free, Individual, Family, Duo, Student). They can listen to songs, like songs, share songs, or follow artists.

Listener Needs

- Access: Needs access to Spotify by creating their account.
- Subscription: Needs options to either subscribe or stay on the free version.
- Music search: Wants to search for music, artists, and genres.
- Recommendation: Listeners expect recommendations related to trending songs, song recommendations of their favourite artists, genres, etc.
- Personalization: Wants to create and edit playlists, like songs, and follow artists they like.

2. Artist: Person who creates content on Spotify. Artists can create their account on Spotify. They can manage their profile and upload music on Spotify as an individual or as a group.

Artist Needs

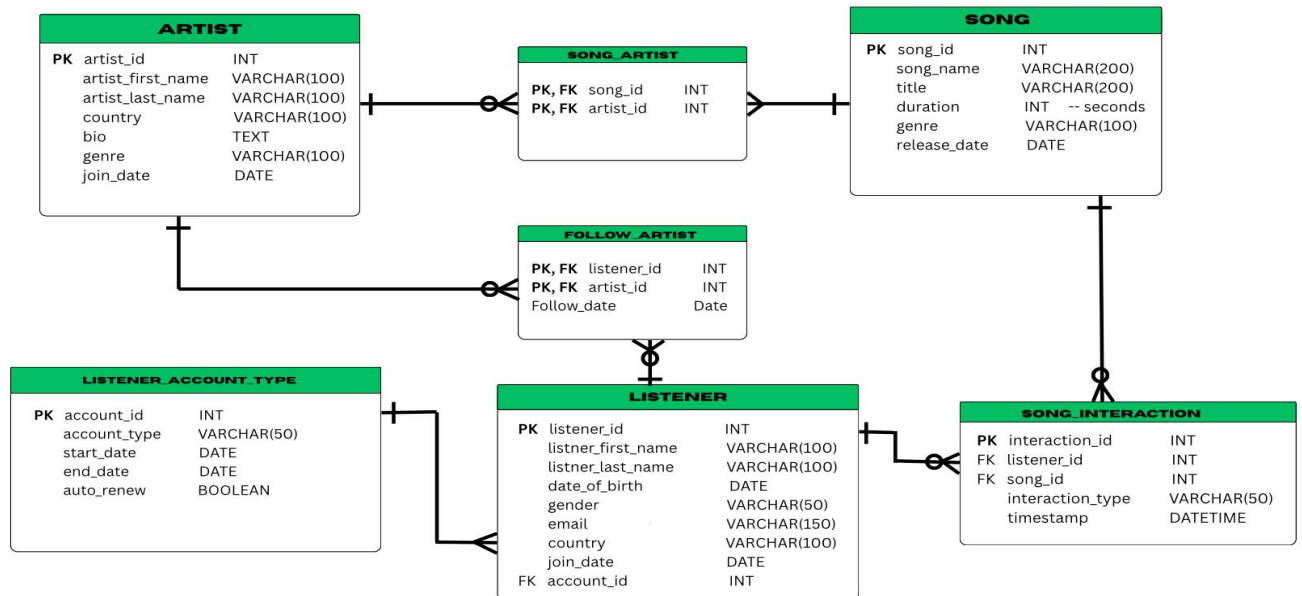
- Access: Needs to create their artist profile on Spotify.
- Profile management: Artists want to build and update their profile with image, genre, bio, description, etc.
- Upload content: Want to be able to upload new music or edit/delete content.
- Analytics: Wants to know the number of followers they have, and how many likes, shares, and plays their songs have.

3. Spotify Team(Analysts/Data Scientists/Engineers/Accountant etc):

These are the people who maintain, develop, and improve the Spotify platform. They use the Spotify data to track billing, develop ML models to improve song recommendations, and keep a check on policy violations, etc.

Note: For this project, I will keep my main focus on listeners and Artists

Entity Relationship Diagram



Note: I have created this ERD diagram using Canva

Relational/Logical database Schema Representation of Spotify

Table 1: The **Artist** table stores information about all the artists. The primary key for this table is artist id

```
ARTIST(  
  artist_id Primary Key INT,  
  artist_first_name VARCHAR(100),  
  artist_last_name VARCHAR(100),  
  country VARCHAR(100),  
  bio TEXT,  
  genre VARCHAR(100),  
  join_date DATE)
```

Table 2: The **Song** table stores information about all the songs. The primary key for this table is **song_id**

```
SONG(  
    song_id Primary Key INT,  
    song_name VARCHAR(200),  
    title VARCHAR(200),  
    duration INT,           -- in seconds  
    genre VARCHAR(100),  
    release_date DATE)
```

Table 3: The **Song_artist** table stores information about all the songs and their artists. This is a junction table representing a many-to-many relationship between songs and artists. A song can have multiple artists, and an artist can sing multiple songs.

```
SONG_ARTIST(  
    song_id Primary Key, Foreign Key INT → SONG(song_id),  
    artist_id Primary Key, Foreign Key → ARTIST(artist_id))
```

Table 4: The **Listener_account_type** table stores information about the kind of account (family, individual, free) each **account_id** is associated with. The Listener table will be joined on account type with this table. Each account will have a type. In a family account, there will be multiple listeners linked to the same account number.

```
LISTENER_ACCOUNT_TYPE(  
    account_id Primary Key INT,  
    account_type VARCHAR(50),    -- e.g., Individual, Family, free  
    start_date DATE,  
    end_date DATE,  
    auto_renew BOOLEAN)
```

Table 5: The **Listener** table stores information about all the Listeners with unique **listener_id**

```
LISTENER(  
    listener_id Primary Key INT,  
    listner_first_name VARCHAR(100),  
    listner_last_name VARCHAR(100),  
    date_of_birth DATE,  
    gener VARCHAR(50),
```

email VARCHAR(150),
country VARCHAR(100),
join_date DATE,
account_id Foreign Key INT →LISTENER_ACCOUNT_TYPE(account_id))

Table 6: The Song_interaction table stores information related to all kinds of interactions listeners perform with different songs. Each interaction (play/like/share) by a listener with a song will have a different interaction_id.

SONG_INTERACTION(
interaction_id Primary Key INT,
listener_id Forign Key INT → LISTENER(listener_id),
song_id Forign Key INT → SONG(song_id),
interaction_type VARCHAR(50), -- Play / Like / Share
timestamp DATETIME)

Table 7: The Follow_Artist table has information related to listeners following different artists and artists being followed by multiple listeners. This is a junction table representing a many-to-many relationship between listener_id and artist_id.

FOLLOW_ARTIST(
listener_id Primary Key, Forign Key INT → LISTENER(listener_id),
artist_id Primary Key, Foreign Key INT → ARTIST(artist_id)
follow_date DATE)

Spotify Database Tables: Data Variables, Semantics, and Constraints

Table	Data Variable	Semantics	Constraints
ARTIST	artist_id	Unique identifier for each artist	Primary Key, INT, Not Null, auto_increament
	artist_first_name	Artist's first name	VARCHAR(100), Not Null
	artist_last_name	Artists last name	VARCHAR(100), Not Null
	country	Country of origin of the artist	VARCHAR(100)
	bio	Short biography or description of the artist	TEXT
	genre	Main genre of the artist	VARCHAR(100)
	join_date	Date the artist joined the platform	DATE

SONG	song_id	Unique identifier for each song	Primary Key, INT, Not Null, auto_increment
	song_name	Name of the song	VARCHAR(200), Not Null
	title	Full title of the song (if different from name)	VARCHAR(200)
	duration	Length of the song in seconds	INT, Not Null, >0
	genre	Genre of the song	VARCHAR(100)
	release_date	Release date of the song	DATE
SONG_ARTIST	song_id	Identifier of the song	Composite Primary Key (with artist_id), Foreign Key → SONG(song_id), Not Null
	artist_id	Identifier of the artist	Composite Primary Key (with song_id), Foreign Key → ARTIST(artist_id), Not Null
LISTENER_ACCOUNT_TYPE	account_id	Unique account identifier	Primary Key, INT, Not Null, auto_increment
	account_type	Type of account (Free, Individual, Family, Duo, Student)	VARCHAR(50), Not Null
	start_date	Account start date	DATE
	end_date	Account end date	DATE
	auto_renew	Whether account auto-renews	BOOLEAN, Default False
LISTENER	listener_id	Unique identifier for each listener	Primary Key, INT, Not Null, auto_increment
	listener_first_name	Listener's first name	VARCHAR(100), Not Null
	listener_last_name	Listener's last name	VARCHAR(100), Not Null
	Date_of_birth	Listener's age	DATE
	gender	Listener's gender(Male, Female, others)	VARCHAR(50), Not Null
	email	Listener's email address	VARCHAR(150), Unique, Not Null
	country	Listener's country	VARCHAR(100)
	join_date	Date listener joined the platform	DATE
	account_id	Listener's account identifier(if they are Free, Individual, Family, Duo, Student)	Foreign Key → LISTENER_ACCOUNT_TYPE(account_id), Not Null
SONG_INTERACTION	interaction_id	Unique identifier for each interaction	Primary Key, INT, Not Null, auto_increment
	listener_id	Listener performing the interaction	Foreign Key →

	song_id	Song being interacted with	LISTENER(listener_id), Not Null Foreign Key → SONG(song_id), Not Null
	interaction_type	Type of interaction (Play, Like, Share, skip)	VARCHAR(50), Not Null
	interaction_timestamp	Date and time of interaction	DATETIME, Not Null
FOLLOW_ARTIST	listener_id	Listener following the artist	Composite Primary Key (with artist_id), Foreign Key → LISTENER(listener_id), Not Null
	artist_id	Artist being followed	Composite Primary Key (with listener_id), Foreign Key → ARTIST(artist_id), Not Null
	follow_date	date_when_followed	DATE

User Interaction Use Cases for the Spotify Database

Use Case 1: An Artist Releases a New Song

Action: An artist publishes a new track or collaboration to the platform.

Database Interaction: INSERTION is performed into two tables:

1. A new record is added to the SONG table (fields: song_name, duration, genre, release_date).
2. New records are added to the SONG_ARTIST junction table (linking song_id to artist_id).

Use Case 2: A Listener Interacts with a Song (Play, Like, Skip)

Action: A listener initiates a play, likes a song, shares it, or skips it.

Database Interaction: A new row is INSERTED into the SONG_INTERACTION table for every distinct action. The new record includes interaction_id, listener_id, song_id, interaction_type (e.g., "Play," "Like"), and interaction_timestamp.

Use Case 3: An Artist Monitors Performance Metrics

Action: An artist checks their dashboard to view real-time statistics on song popularity and fan engagement.

Database Interaction: ANALYTICAL QUERIES are executed across multiple tables:

1. Song Likes: A COUNT query on the SONG_INTERACTION table filters for the artist's songs where interaction_type = 'Like'.
2. Followers: A COUNT query on the FOLLOW_ARTIST table filters by the artist's artist_id.

Use Case 4: A New Listener Joins the Platform

Action: A new user signs up for the music service (e.g., "Free" or "Individual" account).

Database Interaction: INSERTION is performed sequentially into two tables:

1. A new record is added to the LISTENER_ACCOUNT_TYPE table (captures account_type, start_date, auto_renew).
2. A new record is added to the LISTENER table (populates personal details and references the new account_id).

Use Case 5: A Listener Follows an Artist

Action: A listener explicitly chooses to follow an artist to receive updates or notifications.

Database Interaction: A record is INSERTED into the FOLLOW_ARTIST junction table. The new record links the listener_id, the artist_id, and the follow_date.